

Roteiro do vídeo demonstrativo (máx. 15 min)

Iago Roberto Esmério Almeida — PPGCC/UFPI

Fala

Gancho (transição)

comando

Requisitos do enunciado: (1) apresentação inicial em 2 min; (2) demonstração ao vivo no terminal em 8 min, com o Controlador-v2 e a árvore no Mininet, o `ping -f` levando o `top` a 100% de CPU e o `cbench` mostrando responses/sec; (3) análise dos resultados e defesa das 3 refatorações em 5 min.

Bloco	Tempo	Acumulado
1. Apresentação	2:00	2:00
2.1 Controlador + árvore no Mininet	1:30	3:30
2.2 <code>pingall</code> + <code>ping -f</code> + <code>top</code> a 100%	2:45	6:15
2.3 <code>cbench</code>	2:45	9:00
3.1 Gráficos (diagnóstico)	2:45	11:45
3.2 Defesa das 3 refatorações	3:15	15:00
Encerramento	2:00	17:00

Grave com folga: o ideal é fechar em ~14:30. O bloco 3 explica cada gráfico e defende as refatorações, e o encerramento ficou mais completo; para compensar, a demonstração foi encurtada para ~7 min. O passo 4 do `cbench` (refatoração ao vivo) só entra se o ensaio fechar abaixo de 14 min, e os trechos marcados como opcionais são os primeiros a cortar. **Atenção ao tempo:** a tabela acima usa estimativas realistas (≈ 155 palavras/min) e **fecha em ~17 min, acima do limite de 15**. O bloco 3 + encerramento têm ≈ 1.280 palavras de fala (≈ 8 min). Antes de gravar, é preciso cortar cerca de 2 min. Candidatos, nesta ordem: (1) o comentário opcional do `exp4_cpu`; (2) a série temporal do `exp2_ping_flood`, que já foi vista ao vivo; (3) o painel (a) do `exp3_cbench`; (4) o parágrafo de reflexão sobre SDN ou o de limitações do encerramento; (5) falas da demonstração que repetem o que os gráficos mostram depois.

Neste arquivo, as **falas** aparecem como citações (>), os **comandos** em blocos de código e as indicações de **onde apontar** na tela em *itálico*. As frases em **negrito** no fim de cada bloco são os **ganchos**: servem para amarrar uma parte à seguinte, sem silêncio enquanto você troca de janela.

0. Preparação (antes de apertar o REC)

Ensaio obrigatório

O README avisa que a carga com `ping -f -l` (preload) **não foi testada**. Faça um ensaio completo do bloco 2.2 antes de gravar e confira se o `top` passa de 97%. Se não passar, use o plano B (seção 2.2, passo 5).

Limpeza do ambiente

```
cd ~/TA-Network/Trabalho      # pasta do trabalho (todos os comandos partem daqui)
sudo -v                       # deixa o sudo em cache, para não digitar senha no
↪ vídeo
sudo mn -c                     # remove restos de execuções anteriores do Mininet
ss -ltnp | grep 6653           # NÃO deve mostrar nada (porta livre)
killall -f Controlador         # se mostrou algo, mate o processo e confira de novo
```

Layout dos terminais

Deixe três terminais lado a lado, com fonte grande. Em **cada um** deles, entre na pasta do trabalho antes de qualquer outro comando:

```
cd ~/TA-Network/Trabalho
pwd                      # deve mostrar /home/iagora/TA-Network/Trabalho
ls                        # deve listar Controlador-v2.py, topologia_arvore.py,
↪  refatoracoes/ ...
```

Terminal	Função
T1 (esquerda)	Controlador
T2 (centro)	Mininet, depois cbench
T3 (direita, em cima)	top / htop

Deixe também abertos, para o bloco 3:

- o artigo/artigo.pdf (ou as figuras em artigo/figuras/*.png);
- o Controlador-v2.py no editor, já posicionado nas funções `recv_exact()`, `handshake()` e `main()` (laço principal com `listen(5)`).

```
cd ~/TA-Network/Trabalho/artigo
explorer.exe artigo.pdf      # no WSL abre no leitor do Windows (no Linux nativo:
↪  xdg-open artigo.pdf)
explorer.exe figuras         # pasta com as figuras exp1_*, exp2_*, exp3_*, exp4_*
cd ~/TA-Network/Trabalho
code Controlador-v2.py refatoracoes/ # editor com o controlador e as refatorações
```

Dica para o bloco 3: abrir os PNGs da pasta `figuras` em tela cheia, um por vez, facilita apontar com o mouse as curvas e os eixos.

1. Apresentação inicial (2 min)

Tela: primeira página do artigo (título, autor, resumo).

Olá! Eu sou Iago Roberto Esmério Almeida, aluno do Programa de Pós-Graduação em Ciência da Computação da UFPI. Este vídeo faz parte da primeira avaliação da disciplina de Tópicos Especiais em Redes de Computadores, com o professor Rayner Gomes Sousa.

O ponto de partida é uma característica central de SDN: como o plano de controle fica separado do plano de dados, o controlador entra no caminho crítico de cada novo fluxo. Se ele for lento, a rede inteira fica lenta. Então a pergunta que guia o trabalho é simples: até onde um controlador aguenta, e qual trecho do código é responsável por cada limite?

O controlador analisado é o `Controlador-v2.py`, um controlador minimalista que implementa o OpenFlow 1.0 direto sobre sockets TCP, em Python puro e com uma única thread. Três decisões de projeto dele vão aparecer o tempo todo neste vídeo: ele varre os sockets por *polling*; faz o handshake de forma síncrona, um switch por vez; e se comporta como um *hub*, respondendo a todo `PACKET_IN` com um `PACKET_OUT` de inundação,

sem instalar regras. Ou seja, todo pacote da rede passa por ele.

Para medir esses limites, são avaliados quatro experimentos no Mininet com Open vSwitch, numa árvore de profundidade 2 e fanout 4, com 5 switches e 16 hosts, e usei também o cbench. O primeiro estressa o handshake com conexões simultâneas; o segundo inunda o controlador de PACKET_IN com `ping -f`; o terceiro mede a vazão bruta no cbench; e o quarto compara o original com três refatorações que implementei, usando `selectors`, `threads` e `asyncio`.

Gancho → E a melhor forma de entender esses limites é vê-los acontecer. Então vamos para o terminal.

2. Demonstração ao vivo no terminal (≈ 7 min)

2.1 Inicialização do controlador e da árvore ($\approx 1:30$)

Passo 1, T1: subir o controlador no modo padrão (verboso).

```
cd ~/TA-Network/Trabalho
python3 Controlador-v2.py
```

Saída esperada: `=== Controlador SDN Didático v2 ... e aguardando conexões dos switches (porta 6653)...`

No primeiro terminal, inicio o controlador. Ele abre um socket TCP na porta 6653, a porta padrão do OpenFlow, com `listen(5)`. Guardem esse número: ele significa que a fila de conexões pendentes comporta no máximo cinco, e isso vai fazer diferença daqui a pouco. Neste modo, ele também imprime uma linha para cada PACKET_IN, o que tem seu custo.

Com o controlador esperando, falta a rede.

Passo 2, T2: subir a árvore no Mininet.

```
cd ~/TA-Network/Trabalho
sudo mn --topo tree,depth=2,fanout=4 \
        --controller remote,ip=127.0.0.1,port=6653 \
        --switch ovsk,protocols=OpenFlow10
```

(Equivalente, de dentro de ~/TA-Network/Trabalho: `sudo python3 topologia_arvore.py`.)

No segundo terminal, subo a árvore no Mininet: cinco switches Open vSwitch apontando para o controlador. Agora reparem no que aparece no primeiro terminal.

Passo 3, T1: apontar o log do handshake. Role o terminal até o início, se for preciso.

Este é o Experimento 1 acontecendo ao vivo. O log é estritamente sequencial: para cada switch aparecem a conexão TCP, o HELLO, o FEATURES_REQUEST e o FEATURES_REPLY com o DPID e o número de portas. Só quando um switch termina é que o próximo é aceito. Cada handshake leva de 2 a 4 milissegundos, e os cinco concluíram.

O “Handshake falhou” no início não é um switch, é a sonda com que o Mininet testa a porta. E o “PORT_STATUS ignorada” é uma mensagem que chegou enquanto o controlador estava preso negociando com outro switch.

Aqui funcionou porque o Mininet cria os switches um de cada vez. Quando as conexões chegam juntas, o `handshake()` bloqueia o laço inteiro, e com 16 switches a latência passa de 4 segundos. Vou mostrar isso no gráfico, na análise.

Logo depois aparecem linhas de `PACKET_IN` (tráfego IPv6 de descoberta de vizinhos dos hosts).

E, sem que eu tenha mandado nada, já começam os `PACKET_IN`: como o hub não instala regras, até o tráfego de descoberta de vizinhos dos hosts sobe para o controlador.

Gancho → Se até esse tráfego de fundo passa pelo controlador, a pergunta natural é: o que acontece quando eu realmente carrego a rede? É o Experimento 2.

2.2 pingall, ping -f e o top a 100% (≈ 2:45)

Passo 1, T3: abrir o top só com o processo do controlador.

```
cd ~/TA-Network/Trabalho
top -d 1 -p $(pgrep -f Controlador-v2.py)
```

(Se preferir o `htop`, que é visualmente mais claro: `htop -p $(pgrep -f Controlador-v2.py)`.)

Para acompanhar a carga, deixo o `top` no terceiro terminal, monitorando só o processo do controlador. Notem que, mesmo em repouso, ele já gasta alguns por cento de CPU: o laço acorda a cada milissegundo para varrer os sockets, tendo ou não algo para fazer.

Antes de estressar, preciso garantir que a rede funciona.

Passo 2, T2 (Mininet): conectividade.

```
mininet> pingall
```

Esperado: Results: 0% dropped (240/240 received).

O `pingall` testa todos os pares de hosts e dá zero por cento de perda. A rede está correta, com cada pacote passando pelo controlador. Agora sim, vamos estressá-lo.

Passo 3, T2: o comando exato do enunciado.

```
mininet> h1 ping -f h16
```

Deixe rodar uns 15 a 20 s, mostrando o `top`, e depois pare com **Ctrl+C**.

Este é o comando do enunciado: `h1 ping -f h16`, um ping em modo de inundação entre as duas pontas da árvore. Olhem o `top`: a CPU sobe, mas para em torno de 20 a 30%, e não em 100%. No experimento, a média foi de 23%.

Por quê? Porque o `ping -f` é autorregulado: só envia um novo pacote quando chega a resposta do anterior, ou a cada 10 milissegundos. Passando pelo controlador em três switches, o RTT fica em torno de 9 milissegundos, o que limita a carga a pouco mais de 100 pacotes por segundo. Ou seja, a própria lentidão do controlador segura a carga.

Ao parar, mostre as estatísticas (`packets transmitted, % packet loss, rtt`).

Então, para levá-lo de fato à exaustão, preciso manter vários pacotes em voo ao mesmo tempo.

Passo 4, T2: ping -f com preload, para chegar a 100%.

```
mininet> h1 ping -f -l 64 h16
```

Deixe rodar 30 a 40 s com a câmera no `top` e no T1 (o log rolando sem parar). Pare com **Ctrl+C**.

Com a opção `-l 64`, o ping mantém 64 pacotes em voo em vez de um. E agora o `top` mostra o controlador em 100% de um núcleo: ele está saturado. No experimento de saturação, isso acontece a partir de cerca de 3 mil pings por segundo. Como cada ping gera uns 10 `PACKET_IN`, por causa da inundação nos switches do caminho, isso equivale a cerca de 30 mil `PACKET_IN` por segundo. Esse é o teto do controlador.

E o tráfego? O excedente se acumula nas filas, o RTT sobe de 9 milissegundos para mais de um segundo e a perda cresce. Mas os switches não desconectam: o controlador segue respondendo aos `ECHO_REQUEST` no mesmo laço. A degradação aparece como perda e latência, não como queda do canal OpenFlow.

Ao parar, mostre a perda e o RTT no resumo do ping.

Passo 5 (plano B, só se o passo 4 não passar de ~95% no ensaio). Injeção em taxa fixa, igual ao `exp2_packet_in.py` taxa: 16 pings simultâneos de 250 req/s cada (≈ 4 mil ICMP/s), que param sozinhos em 40 s.

```
mininet> py [net.get('h%d' % i).popen('ping -q -i 0.004 -w 40 10.0.0.%d' % (17 - i))  
↪ for i in range(1, 17)]
```

Como o ping `-f` se autorregula, faço o mesmo que no experimento de saturação: dezesseis pings em paralelo, em taxa fixa, somando cerca de 4 mil pacotes por segundo. E o `top` vai a 100%.

Se precisar interromper antes dos 40 s: `mininet> sh pkill -f "ping -q -i"`.

Passo 6: confirmar que os switches seguem conectados (opcional, 10 s).

```
mininet> sh ovs-vsctl show | grep is_connected
```

Esperado: 5 linhas `is_connected: true`.

E aqui está a confirmação: os cinco switches continuam conectados.

Passo 7: encerrar o Mininet e o controlador.

```
mininet> exit
```

No T1: **Ctrl+C**.

Gancho → O Mininet mostrou o limite com tráfego realista, mas ali o gargalo se mistura com o custo dos switches e dos hosts. Para medir só o controlador, isolado, a ferramenta padrão é o `cbench`. É o Experimento 3.

2.3 cbench: vazão em responses/sec ($\approx 2:45$)

Passo 1, T1: subir o controlador em modo silencioso.

```
cd ~/TA-Network/Trabalho
python3 Controlador-v2.py --quiet
```

Reinício o controlador com `--quiet`, que desliga o print por mensagem. Assim meço a arquitetura do controlador, e não a velocidade do terminal.

Passo 2, T2: o comando exato do enunciado, com 8 switches.

```
cd ~/TA-Network/Trabalho
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 8
```

(Se o `cbench` não estiver no `PATH`, use o binário da pasta `ferramentas/`, a partir de `~/TA-Network/Trabalho`: `./ferramentas/cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 8`.)

Esperado: falha imediata com `make_tcp_connection: connect: Operation now in progress` e `make_nonblock_tcp:: returned -1`.

Este é o comando do enunciado: 8 switches emulados mandando `PACKET_IN`, em 10 laços de 10 segundos. E ele falha na hora.

Lembram do `listen(5)` e do handshake um switch por vez? É isso. O `cbench` abre as 8 conexões ao mesmo tempo, com `connect()` não bloqueante, e trata como erro qualquer conexão que não se complete na hora. Com o controlador preso num handshake e só cinco lugares na fila, a oitava fica pendente e o teste aborta, em todas as execuções. É o gargalo do Experimento 1, agora impedindo o próprio benchmark de rodar.

Se o controlador ficou travado/girando (veja o `top`), reinicie-o no T1 (`Ctrl+C` e `python3 Controlador-v2.py --quiet`).

Então a pergunta passa a ser: qual o maior número de switches que ele aceita?

Passo 3, T2: com 7 switches, o maior N que funciona.

```
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 7
```

Leva ~100 s. Use esse tempo para falar, mostrando o `top` (no T3, refaça `top -d 1 -p $(pgrep -f Controlador-v2.py)`, porque o PID mudou).

Com 7 switches o teste completa, então essa é a vazão máxima que consigo relatar. Cada linha que aparece é um laço de 10 segundos, com o número de respostas por segundo. E no `top` a CPU fica perto de 90%.

Enquanto o teste roda, vale entender o que está sendo medido. No modo padrão do `cbench`, o de latência, cada switch emulado mantém uma única requisição pendente: manda um `PACKET_IN` e espera a resposta para mandar o próximo. Para o controlador, cada mensagem custa duas chamadas de `recv` e um `sendall`, mais o giro do *polling* sobre os sockets vazios, que lança uma exceção `BlockingIOError` a cada volta. Esse custo por mensagem é o que vai definir o resultado.

Ao terminar, aponte a linha final:

```
RESULT: 7 switches 9 tests min/max/avg/stdev = ... responses/s
```

E o resultado: cerca de 108 mil respostas por segundo em média. São 9 testes porque o primeiro laço é aquecimento e é descartado. Para ter uma referência, na literatura o

Beacon passa de um milhão de respostas por segundo com uma única thread, e o ONOS chega a centenas de milhares.

Passo 4 (opcional, só se o ensaio fechar abaixo de 14 min; $\approx$ 45 s): a refatoração com `selectors` aguenta 8 switches.

T1: Ctrl+C e:

```
cd ~/TA-Network/Trabalho
python3 refatoracoes/Controlador-v3-selectors.py
```

T2:

```
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 3 -s 8
```

E se o problema é a arquitetura, mudar a arquitetura deveria resolver. Aqui está a refatoração baseada em `selectors` rodando o mesmo teste com 8 switches: ele roda normalmente, a cerca de 125 mil respostas por segundo. Reduzi para 3 laços só para caber no tempo do vídeo.

Ao final, Ctrl+C no T1 e confira que a porta ficou livre (`ss -ltnp | grep 6653`).

Gancho → Até aqui vimos os sintomas ao vivo: o handshake sequencial, a CPU a 100% e o `cbench` falhando com 8 switches. Agora vamos aos números das rodadas completas: primeiro, para fechar o diagnóstico, e depois para defender as três refatorações que proponho.

3. Análise dos resultados e defesa arquitetural ($\approx$ 6:00)

Tela: figuras em tela cheia (ou o artigo em PDF), uma por vez. Para cada gráfico: diga em uma frase o que está em cada eixo, depois aponte com o mouse a curva que está descrevendo. O bloco tem duas partes: **3.1**, os gráficos dos Experimentos 1 a 3 ($\approx$ 2:45), e **3.2**, a defesa das três refatorações, com os gráficos do Experimento 4 como apoio visual ($\approx$ 3:15). Se atrasar, corte primeiro o parágrafo do microbenchmark e o gráfico de CPU (`exp4_cpu`).

```
cd ~/TA-Network/Trabalho/artigo
explorer.exe artigo.pdf          # ou: xdg-open artigo.pdf
```

3.1 O diagnóstico: o que os gráficos mostram ($\approx$ 11:00)

Para cada gráfico, a fala segue o mesmo caminho: (1) o que o experimento fez; (2) como ler o gráfico (eixos e cores); (3) o que acontece da esquerda para a direita; (4) por que isso acontece; (5) o que isso significa. Vá apontando com o mouse enquanto fala.

Experimento 1 (`exp1_handshake`), $\approx$ 3:40

Onde apontar: primeiro os eixos (X: switches; Y: tempo em escala log); depois a curva vermelha no painel (a), subindo da esquerda para a direita; no painel (b), o salto vermelho entre 1 e 2 switches; por fim, as curvas azul, rosa e verde embaixo.

Começando pelo Experimento 1, o do handshake. A ideia foi simples: fazer vários switches se conectarem ao controlador exatamente ao mesmo tempo, de um até dezesseis, e medir, com capturas de pacotes, quanto tempo cada um levou para ser atendido. Cada ponto é a média de cinco rodadas, e as barrinhas verticais mostram a variação entre elas.

Os dois painéis têm a mesma leitura. No eixo horizontal está o número de switches chegando juntos. No eixo vertical está o tempo, em milissegundos, e aqui vale um cuidado: a escala é logarítmica, então cada linha de grade vale dez vezes a anterior. Uma distância pequena no gráfico pode ser uma diferença enorme na prática. A cor vermelha é o controlador original, e azul, verde e rosa são as três refatorações.

No painel da esquerda está a latência total do handshake, do primeiro pacote até o switch ser registrado. Seguindo a curva vermelha da esquerda para a direita, ela sobe o tempo todo: cerca de 0,3 segundo com um switch, pouco mais de um segundo com cinco, dois segundos e meio com oito e mais de quatro segundos com dezesseis. A explicação é que o controlador atende um switch de cada vez. Então o último da fila espera o handshake de todos os que chegaram antes dele, e cada switch a mais aumenta a espera de todo mundo.

O painel da direita isola a causa. Ele mede só a espera até o controlador mandar o primeiro HELLO, ou seja, quanto tempo o switch ficou parado na fila antes de ser atendido. Com um switch, essa espera é praticamente zero, menos de um milissegundo. Mas, com apenas dois switches, a curva vermelha dá um salto de mais de cem vezes, para cerca de 200 milissegundos, e chega a um segundo com dezesseis. Esse salto logo no segundo switch é a assinatura do handshake síncrono: enquanto o controlador negocia com um, o outro simplesmente não é atendido. E com dezesseis switches entra o `listen(5)`: a fila de conexões pendentes enche, o sistema deixa de responder a novas conexões, e, em média, nove conexões por rodada foram abortadas e tiveram de ser refeitas.

Agora olhem a parte de baixo do gráfico. As refatorações com `selectors` e `asyncio`, em azul e rosa, ficam praticamente planas, com o HELLO saindo em torno de um milissegundo, não importa quantos switches cheguem. A de `threads`, em verde, sobe um pouco, porque precisa criar uma thread para cada conexão, mas continua ordens de grandeza abaixo do original. Algumas irregularidades no painel da esquerda, como o degrau da curva verde e o pico isolado da rosa em cinco switches, vêm do próprio Open vSwitch, que demora para responder. O painel do HELLO mostra que, nesses casos, o controlador já tinha feito a sua parte.

A conclusão do Experimento 1, então, é que o controlador original não escala em número de conexões, e o motivo não é processamento: é o fato de ele esperar um switch terminar para atender o próximo.

Gancho → Esse é o limite de conexões. Mas e depois que todos os switches estão conectados: quanto tráfego o controlador aguenta?

Experimento 2 (`exp2_ping_flood` e `exp2_saturacao`), $\approx$ 4:40

Onde apontar (`exp2_ping_flood`): as quatro faixas no topo (ocioso, pingall, ping -f, pós-flood); a linha vermelha em cada uma; depois a azul tracejada, acima da vermelha.

O Experimento 2 olha para o tráfego. Este primeiro gráfico é exatamente o roteiro que eu mostrei ao vivo, só que registrado ao longo do tempo. No eixo horizontal está o tempo, em segundos, e no vertical, a CPU, em porcentagem de um núcleo. A linha vermelha é o controlador, e a azul tracejada é o Open vSwitch, o processo que faz o papel dos switches. As linhas pontilhadas verticais dividem o teste em quatro fases.

Na primeira fase, com a rede ociosa, o controlador já fica entre 5 e 10% de CPU, sem nenhum tráfego. Esse é o preço do *polling*: ele acorda o tempo todo para perguntar aos sockets se chegou algo. Quando entra o `pingall`, e depois o `ping -f`, a linha vermelha sobe e se estabiliza num patamar plano, em torno de 23%. Ela não continua subindo, e isso é o mais interessante do gráfico: o `ping -f` só manda um pacote novo quando o anterior volta, então quanto mais lento o controlador, menos carga ele recebe. É um sistema que se autorregula. Reparem também que o Open vSwitch, em azul, gasta ainda mais CPU do que o controlador, porque, sem regras instaladas, todo pacote precisa subir dele para o controlador e voltar. E quando o flood termina, as duas linhas voltam ao repouso.

Onde apontar (`exp2_saturacao`): o eixo X; as curvas vermelha e azul subindo juntas até a faixa rosa; a azul achatando à direita da faixa; a preta saindo do zero na faixa rosa e subindo; o eixo da direita, que é o da curva azul.

Como o `ping -f` se autorregula, ele não serve para encontrar o limite. Então, neste segundo gráfico, eu controlei a carga: injetei pings em taxa fixa, aumentando degrau por degrau. O eixo horizontal é essa taxa, em pings por segundo. O gráfico tem dois eixos verticais. No da esquerda, em porcentagem, estão a CPU do controlador, em vermelho, e a perda de pacotes, em preto. No da direita, em milhares por segundo, estão os `PACKET_IN` que o controlador conseguiu processar, em azul. Vale lembrar que cada ping gera uns dez `PACKET_IN`, porque o hub inunda o pacote e cada cópia que chega a outro switch volta ao controlador.

A leitura tem três momentos. No primeiro, à esquerda, até cerca de 2,6 mil pings por segundo, a vermelha e a azul sobem juntas, quase em linha reta, e a preta fica colada no zero. Isso quer dizer que tudo o que chega é atendido: mais carga, mais trabalho feito, nenhuma perda. A CPU vai de 22 a quase 80%.

O segundo momento é a faixa rosa, entre 2,6 e 3,2 mil pings por segundo. É ali que o comportamento muda: a CPU encosta em 97%, a curva azul para de subir e achata em torno de 30 mil `PACKET_IN` por segundo, e a curva preta sai do zero. O controlador chegou ao seu teto.

No terceiro momento, à direita da faixa, o que se vê é um controlador saturado. Aumentar a carga não aumenta mais o trabalho feito, a azul fica praticamente horizontal, e todo o excedente vira perda, que sobe até 100%. Os pacotes que sobram ficam esperando nas filas, e o tempo de resposta sobe de milissegundos para mais de um segundo. Esse formato, com uma subida, um joelho e um platô, é a assinatura clássica de saturação.

E um detalhe importante: mesmo nesse estado, os switches não se desconectaram. O controlador degradou perdendo pacotes, e não perdendo o controle da rede.

E o que, no código, define esse teto? Medi o custo de cada operação isoladamente. A decodificação com `struct.unpack()`, que seria o suspeito natural, custa só cerca de 1% do tempo de cada mensagem. O que pesa é a E/S: para cada mensagem o controlador faz duas leituras e uma escrita no socket, o que custa uns 3,4 microssegundos, contra apenas 63 nanossegundos por mensagem quando uma única leitura traz 64 mensagens de uma vez. Some-se a isso o custo das voltas vazias do *polling* e a espera ocupada do `recv_exact()` quando uma mensagem chega fragmentada.

Gancho → Se o custo está na forma de conversar com a rede, uma mensagem por vez, o `cbench`, que mede só o controlador, deve mostrar o mesmo padrão.

Experimento 3 (`exp3_cbench`), ≈ 2:50

Onde apontar: no painel (a), as linhas planas e empilhadas, de baixo (1 switch) para cima (7 switches); no (b), as barras subindo, o salto entre 4 e 5, a linha da CPU acompanhando e as marcas “falha” em 8 e 16.

No Experimento 3, o `cbench` faz o papel de vários switches ao mesmo tempo e mede quantas respostas por segundo o controlador consegue dar. Cada teste dura dez laços de dez segundos, e o primeiro laço é descartado como aquecimento.

No painel da esquerda, o eixo horizontal é o tempo do teste e o vertical é a vazão, em milhares de respostas por segundo. Cada linha colorida é um teste com um número diferente de switches. Duas coisas aparecem logo. Primeiro, as linhas são praticamente horizontais: a vazão é estável do começo ao fim do teste, então a medida é confiável. Segundo, elas se empilham em ordem: com um switch, a linha azul fica lá embaixo, perto de 30 mil; com sete, a verde-escura fica no topo, perto de 110 mil.

O painel da direita resume cada teste numa barra, a vazão média, e acrescenta a CPU, na linha tracejada, com o eixo à direita. Da esquerda para a direita, as barras crescem: 30 mil com um switch, 44 mil com dois, 67 mil com quatro, e um salto para 96 mil com cinco. A partir daí, o crescimento fica bem mais lento, chegando a 108 mil com sete. Enquanto isso, a CPU sobe de cerca de 30 até quase 90%.

Por que mais switches dão mais vazão? Porque, no *polling*, o controlador dá voltas pelos sockets perguntando se há dados. Com poucos switches, a maioria das voltas é desperdiçada. Com mais switches, cada volta encontra mais mensagens prontas, e o custo fixo da volta se dilui. E por que o crescimento desacelera? Porque a CPU está chegando ao limite: não há mais folga para transformar em vazão.

E em oito e dezesseis switches não há barra nenhuma, só a palavra “falha”: é o que vimos ao vivo. O `cbench` não consegue nem conectar os switches, e o controlador fica a 100% de CPU girando na espera ocupada do handshake. Ou seja, o gargalo do Experimento 1 reaparece aqui e define o teto do benchmark. Para comparação, controladores de

produção chegam a centenas de milhares ou mais de um milhão de respostas por segundo. Como a aplicação aqui é um simples hub, essa diferença não vem da lógica de controle, vem da arquitetura.

Gancho → Diagnóstico fechado: três gargalos, todos de arquitetura. O handshake síncrono, o *polling* e a E/S de uma mensagem por vez. Agora, a defesa das três propostas que atacam esses gargalos.

3.2 Defesa das três propostas de refatoração (≈ 3:15)

Nesta parte a fala é **conceitual**: sem números. Os gráficos ficam na tela só como apoio visual; aponte as tendências (quem fica acima, quem cai, quem fica plano), sem citar valores.

Se quiser, mostre o código lado a lado:

```
cd ~/TA-Network/Trabalho/refatoracoes
ls                               # of10.py, Controlador-v3-selectors.py, -threads.py,
↪ -asyncio.py
code of10.py Controlador-v3-*.py
```

Tela: deixe o `exp4_vazao_cbench` aberto enquanto defende as três; troque para o `exp4_throughput` só no parágrafo da síntese.

Primeiro, o princípio que une as três. Um controlador SDN é, antes de tudo, um servidor de E/S: passa a maior parte do tempo esperando a rede e trocando mensagens pequenas com muitos switches. A pergunta de projeto, então, não é “como decodificar mais rápido”, mas “como esperar melhor”. Por isso, as três mantêm o hub, para a comparação ser justa, e tratam a rede em lotes: leem tudo o que chegou e respondem de uma vez. O que muda é a filosofia de concorrência.

Onde apontar: a curva azul acima da vermelha e continuando depois do ponto de falha.

A **primeira**, com `selectors`, é a multiplexação orientada a eventos: o controlador deixa de perguntar a cada socket se há algo e passa a ser avisado pelo sistema operacional. O handshake, que congelava o controlador, vira uma sequência de eventos, com cada conexão guardando seu estado. O argumento a favor dela é que resolve a causa, não o sintoma: com uma única thread, sem locks, ela supera o original. Ou seja, o problema nunca foi falta de núcleos, e sim a forma de esperar. O preço é um código em máquina de estados, menos linear. É a que eu recomendo para o controlador como ele é hoje.

Onde apontar: a curva verde caindo à medida que entram mais switches.

A **segunda**, uma thread por switch, se apoia no isolamento: cada switch tem seu fluxo de execução, o código continua sequencial, e um switch lento não contamina os outros. É o caminho mais curto do original até um controlador concorrente. O limite é do Python: por causa do GIL, as threads não rodam em paralelo, e com mensagens pequenas e frequentes o custo de passar a vez entre elas domina. Por isso eu a defendo como degrau, não como destino: trocando threads por processos, a mesma estrutura ganha paralelismo real.

Opcional (corte se estiver atrasado), `exp4_cpu`: aponte a barra verde, a mais alta.

O gráfico de CPU reforça a ideia: mais esforço da máquina não significa mais trabalho útil.

Onde apontar: a curva rosa, estável, abaixo das demais.

A **terceira**, com `asyncio` e fila de eventos, é a mais sólida em arquitetura, porque separa o que o original misturava: receber da rede e decidir o que fazer. Entre os dois há uma fila limitada, que traz extensibilidade, porque novas aplicações, como um firewall, consomem eventos sem tocar na rede; e *backpressure*, porque, se o processamento atrasa, só a leitura daquele switch desacelera. É o desenho do Ryu. O custo que aparece no gráfico é da implementação do *event loop*, não da arquitetura, e um *event loop* mais eficiente, como o `uvloop`, o reduz sem mexer no desenho.

Troque para `exp4_throughput`: as três barras das refatorações juntas, bem acima da do original.

Em síntese, são três prioridades: eficiência, com o `selectors`; simplicidade e isolamento, com as threads; extensibilidade, com o `asyncio`. E este gráfico mostra o que as une: quando as mensagens chegam acumuladas, as três ficam muito acima do original e próximas entre si. Tratar a rede em lotes pesa mais do que o modelo de concorrência. Num controlador de produção, elas se combinariam: um laço de eventos por processo, um processo por núcleo, e filas entre a rede e as aplicações.

Gancho → E isso nos leva às conclusões do trabalho.

Encerramento (≈ 2:00)

Fala também **conceitual**, sem números.

Tela: volte para a primeira página do artigo (ou para a seção de Conclusão).

Para encerrar, volto à pergunta do início: até onde esse controlador aguenta, e qual trecho do código é responsável por cada limite?

Cada limite nasce de uma escolha de arquitetura. O de conexões vem da espera síncrona: um switch de cada vez. O de tráfego vem da espera ativa: processamento gasto perguntando à rede se há algo novo. E o de vazão vem da granularidade da E/S: uma mensagem por vez. Nenhum está na lógica de controle, nem na decodificação dos pacotes, que é onde a intuição apontaria primeiro. Daí a lição metodológica: medir antes de otimizar, porque o gargalo raramente está onde se imagina.

Esses gargalos levaram controladores como o Ryu e o ONOS a convergirem para o mesmo desenho: eventos, E/S em lote e filas entre a rede e as aplicações. Este trabalho fez esse caminho em pequena escala, com uma nuance: concorrência não é sinônimo de desempenho. O modelo de concorrência precisa combinar com o custo por mensagem e com o regime de carga.

Há uma reflexão mais ampla sobre SDN. Centralizar o controle simplifica a rede, mas cria um ponto crítico: o desempenho do controlador deixa de ser detalhe de implementação e vira propriedade da rede inteira. E um bom projeto não é só o que aguenta mais, é o que

degrada com elegância: mesmo saturado, o original perdeu pacotes, mas não perdeu os switches.

O trabalho tem limites: um hub é o caso extremo, em que todo pacote passa pelo controle, e o ambiente foi virtualizado. Os próximos passos são instalar regras de fluxo, buscar paralelismo real além do GIL e repetir os experimentos em hardware físico.

Fica, então, a mensagem principal: em SDN, o controlador está no caminho crítico da rede, e a forma como ele espera e conversa com a rede importa mais do que a lógica que ele executa. Obrigado pela atenção!

Problemas comuns durante a gravação

Sintoma	Causa provável	O que fazer
Controlador: Address already in use	Processo antigo na 6653	<code>kill -f Controlador; ss -ltnp grep 6653</code>
Mininet não sobe / interfaces duplicadas	Restos da execução anterior	<code>sudo mn -c</code>
<code>top -p</code> vazio ou com erro	PID mudou ao reiniciar o controlador	Rode o <code>top -p \$(pgrep -f ...)</code> de novo
<code>top</code> não chega a 100% com <code>-l 64</code>	O terminal não acompanha o volume de prints, ou a carga é baixa	Aumente para <code>-l 128</code> ou use o plano B (passo 5)
<code>cbench -s 7</code> também falha	Controlador ficou girando após o <code>-s 8</code>	Reinicie o controlador antes de cada execução do <code>cbench</code>
<code>cbench: command not found</code>	Fora do PATH	<code>cd ~/TA-Network/Trabalho e ./ferramentas/cbench ...</code>
No such file or directory ao rodar <code>python3 ...</code>	Terminal fora da pasta do trabalho	<code>cd ~/TA-Network/Trabalho e confira com <code>pwd</code></code>

Depois de gravar

1. Suba o vídeo (YouTube não listado ou Google Drive com acesso de visualização) e confira o link numa janela anônima.
2. Coloque o link no `artigo.tex`, no lugar dos dois `\todo{link público}` / `\todo{public link}` do resumo e do abstract, e recompile: `cd ~/TA-Network/Trabalho/artigo && latexmk -pdf artigo.tex`.
3. Confira a duração final: no máximo 15:00.